Инструкция по взаимодействию с Gitverse Lab

Для работы с платформой TalentBridge необходимо использовать платформу Gitverse Lab.

Процесс создания задания на платформе GitVerse lab

1. **Подготовка структуры**

Сначала определите, **в каком модуле** будет создано задание и как оно впишется в учебную цепочку. В документации GitVerse задание описывается как конкретная задача внутри модуля, а пример строится по схеме «модуль → задания».
На этом этапе важно понимать, что задание – это не просто отдельная карточка, а часть более крупной структуры обучения, где участники получают одинаковую стартовую точку.

Ссылка на SmartClass: https://gitverse.ru/smartclass

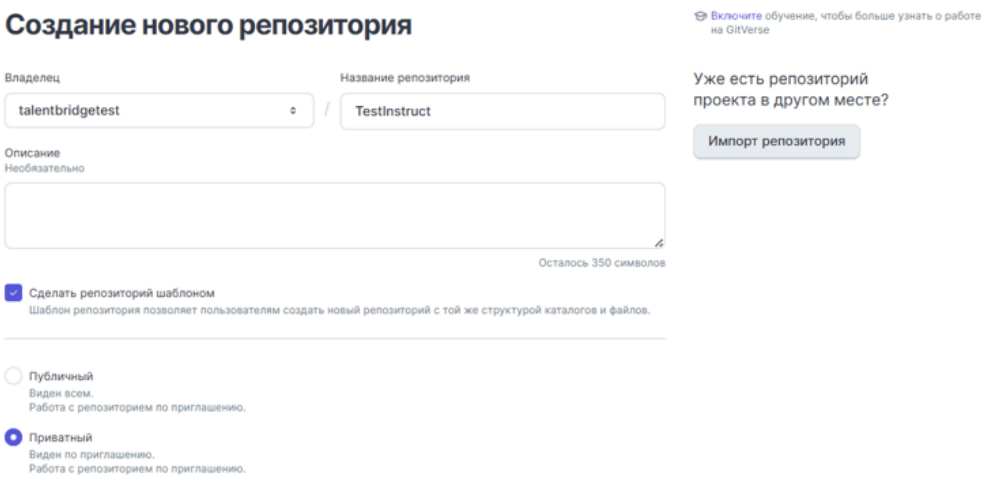
Сперва потребуется создать организацию (если её ещё нет), после чего создать модуль (например, TalentBridge)

2. **Создание репозитория-шаблона**

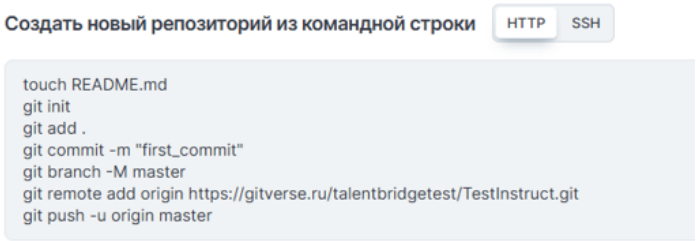
Для задания нужен **репозиторий-шаблон**, так как именно из него GitVerse формирует основу для заданий и потом создает персональные копии для участников.
Чтобы подготовить шаблон, нужно перейти в «Мои репозитории», создать новый репозиторий, указать название, отметить его как шаблон и выбрать приватность – публичную или приватную.
Рекомендуется сразу добавить README, а при необходимости – описание и .gitignore.
Также репозиторий можно импортировать из других git-сервисов.

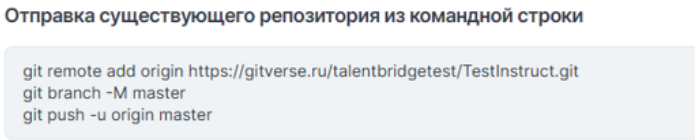
Также можно импортировать уже существующие репозитории из другого места.

Можно создать файл для автопроверок и поместить его в .gitverse.

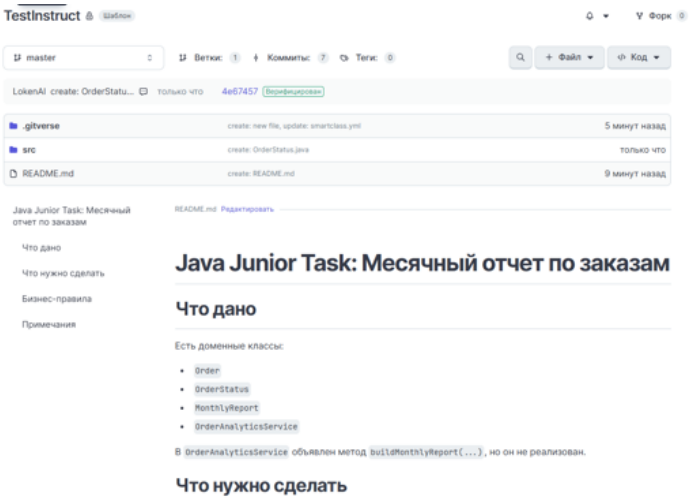


Создание репозитория-шаблона





Примеры команд для работы с созданным репозиторием



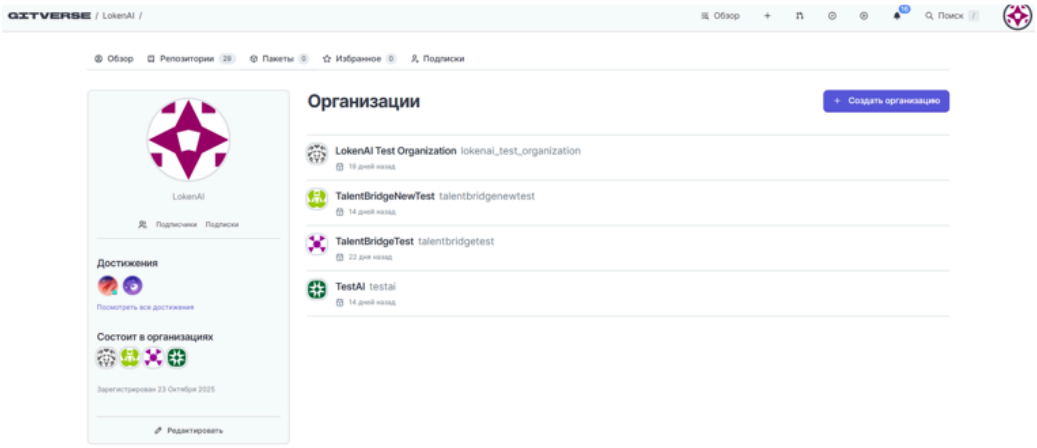
Пример созданного репозитория-шаблона

3. **Настройка типа задания**

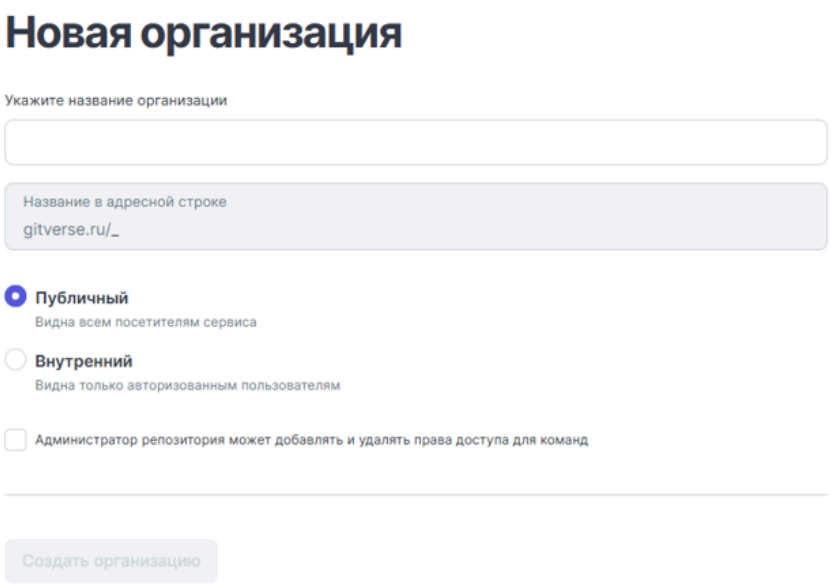
После этого в нужном модуле GitVerse Lab необходимо выбрать **«Новое задание»** и заполнить основные параметры.

Здесь выбирают название, тип задания, уровень приватности и сценарий работы: **индивидуальный** или **групповой**.
Это важный этап, потому что тип задания определяет, как участники будут работать: самостоятельно или в команде, а также как будет устроен доступ к результатам.

Стоит отметить, что можно сначала создать организацию, а затем в этой организации создать **репозиторий-шаблон**, чтобы избежать дополнительных настроек с переопределением принадлежности репозитория.



Создание новой организации

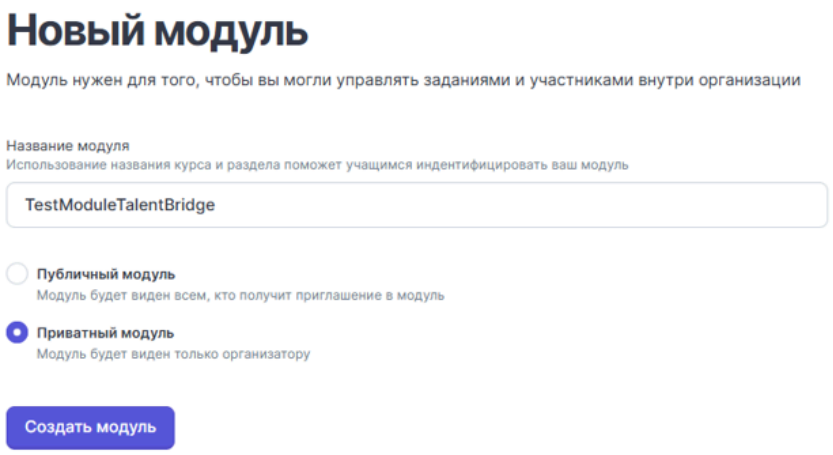


Выбор параметров доступа к организации

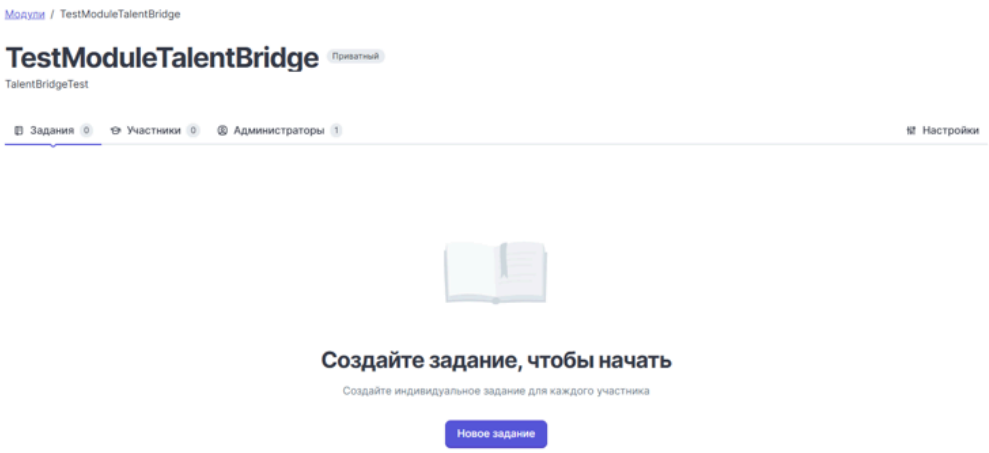
4. **Выбор приватности модуля**

Приватность влияет на видимость задания в модуле и на то, кто увидит его содержимое. **Публичное** задание видно всем участникам модуля.

Приватное скрыто из общего списка и доступно только по прямой ссылке.
Если задание еще заполняется, лучше использовать приватный режим. Если задача должна быть видна всем в рамках курса, подходит публичный режим.



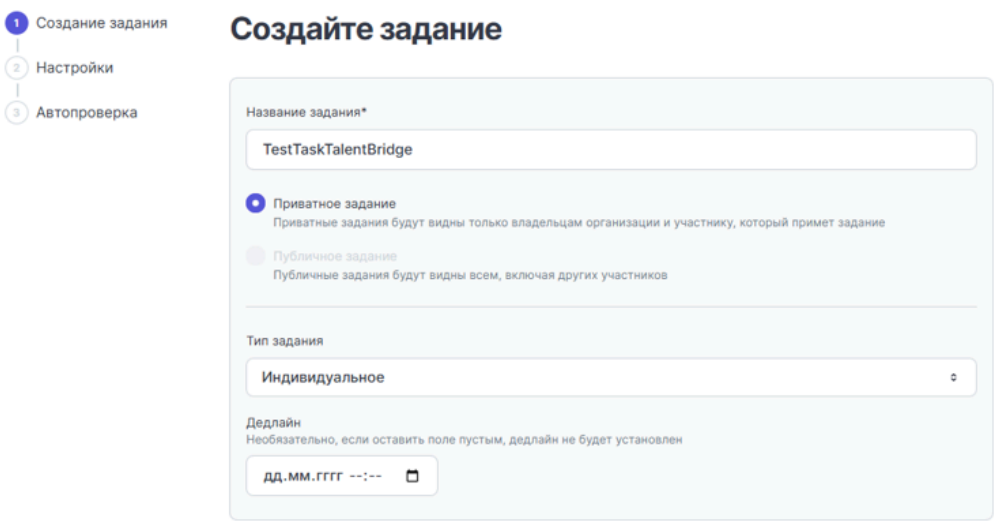
Выбор приватности модуля



Общий вид созданного пространства

5. **Настройка дедлайна и типа задания**

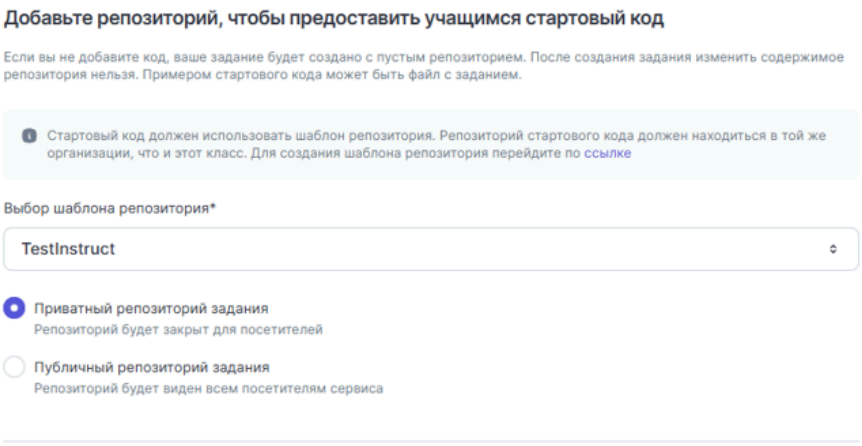
Далее задают **срок выполнения** и при необходимости включают жесткость дедлайна. Дедлайн можно не ставить вообще, а если он нужен, то жесткий вариант ограничивает возможность вносить изменения после его окончания.
На этом этапе стоит особенно внимательно проверить сроки, потому что они напрямую влияют на правила сдачи и редактирования задания. Также можно создать либо индивидуальное, либо групповое задание.



Выбор дедлайна и типа задания

6. **Привязка шаблона**

Следующий шаг – выбрать **репозиторий-шаблон**, который был подготовлен заранее. Именно он станет основой для создания снепшота, а затем – индивидуальных или командных копий для участников.
Это ключевой технический момент: если шаблон подготовлен неправильно, у участников сразу возникнут проблемы со стартом работы.

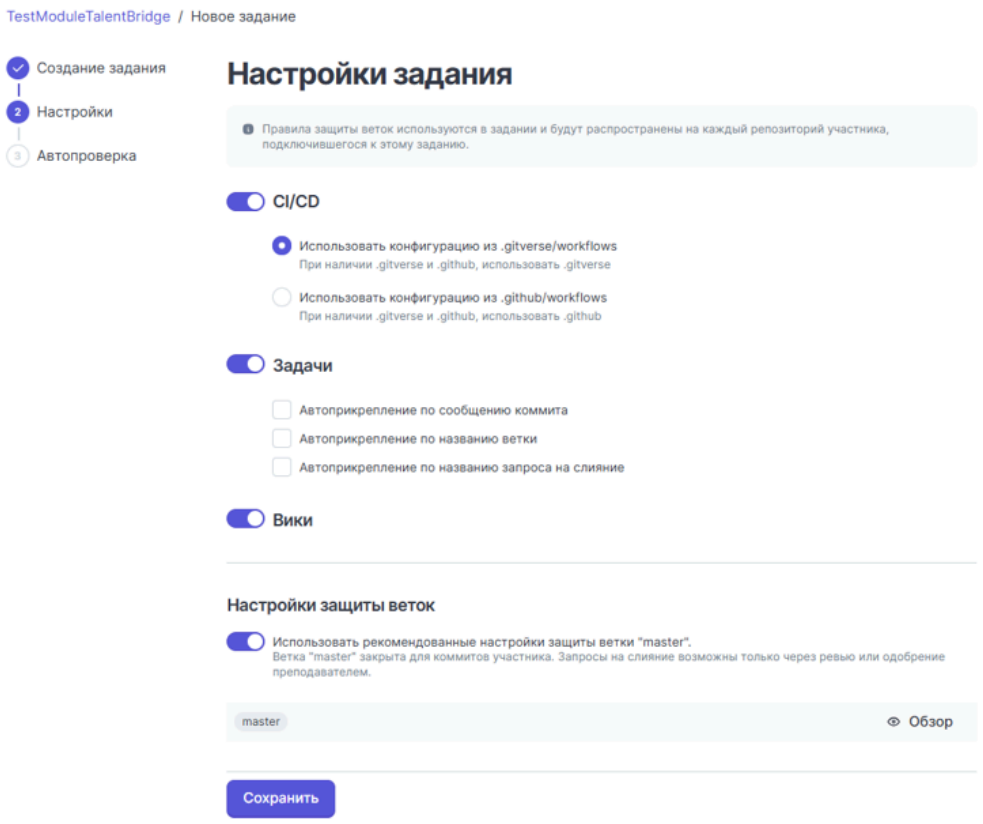


Выбор репозитория-шаблона и приватности

7. **Создание и дополнительные настройки**

После заполнения основных параметров необходимо нажать **«Создать задание»**.
Затем можно включить или отключить дополнительные возможности, в зависимости от потребностей – **CI/CD**, **задачи**, **вики**, **защиту веток**.
Эти функции полезны для более сложных сценариев: автопроверки, документации, управления работой через задачи и контроля над изменениями в ветках.

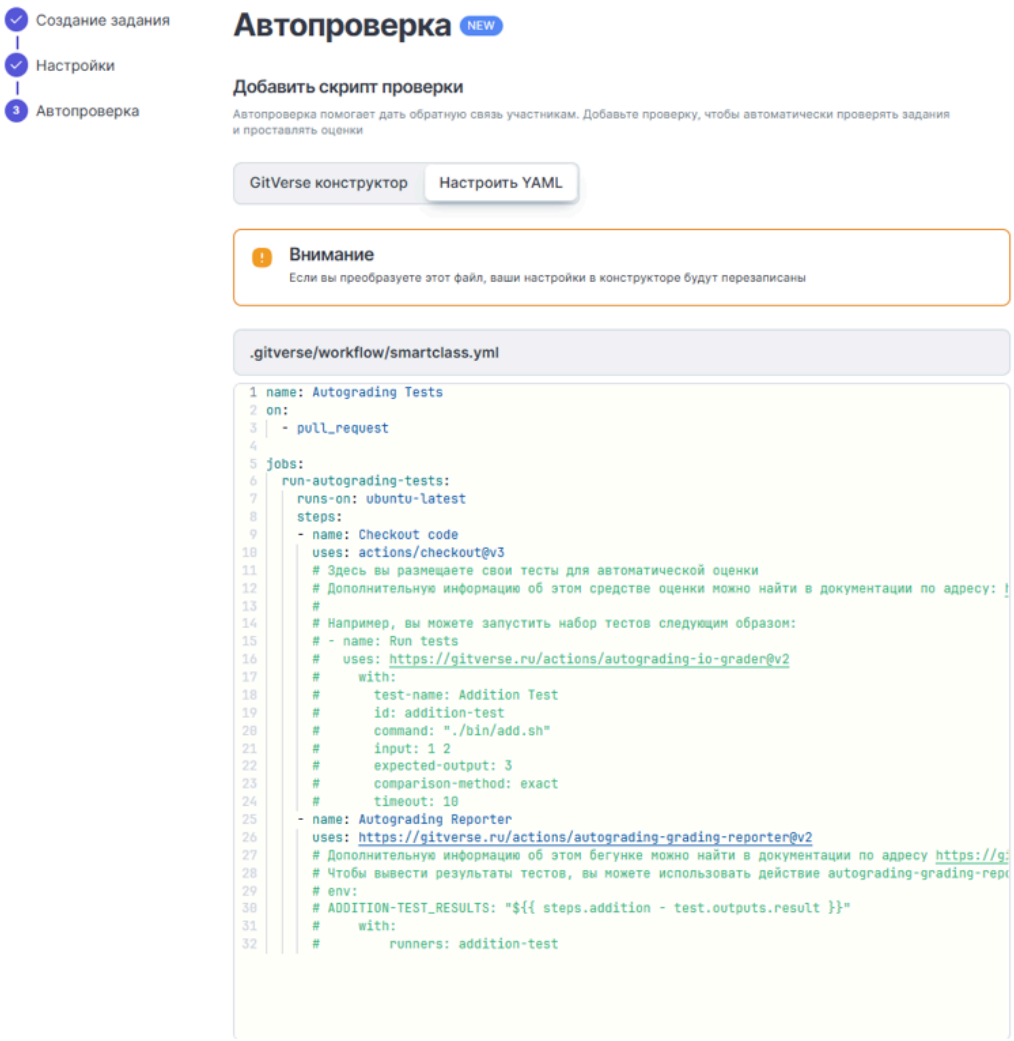
Можно автоматизировать проверку задания, настроить защиту ветки master от изменений со стороны участника, выполняющего задание - то есть изменения он может проводить только в отдельных файлах.



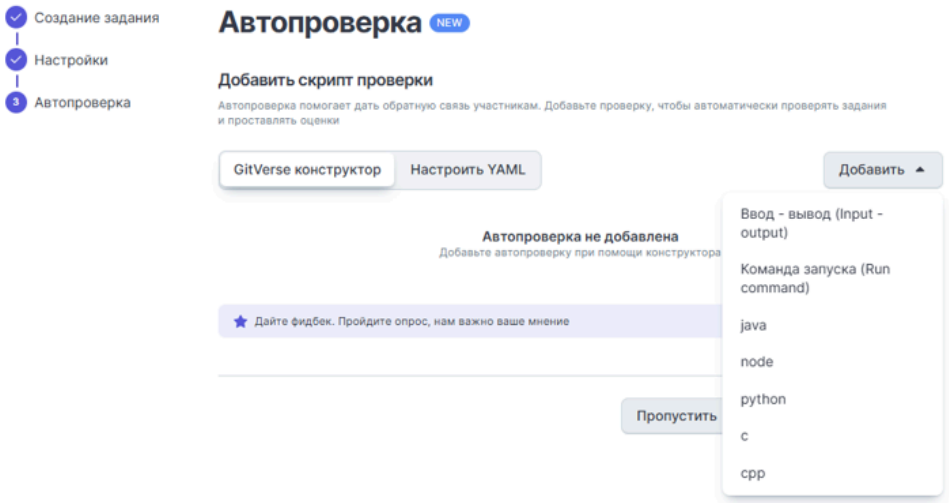
Выбор настроек для задания

8. **Сохранение и автопроверка**

Когда дополнительные параметры выбраны, настройки сохраняют, а шаг с автопроверками можно **пропустить** и вернуться к нему позже. Также можно создать файл автопроверок при создании репозитория-шаблона
Это удобно, если сначала нужно запустить задание, а потом уже донастроить тесты и проверку.



Базовый шаблон файла автоматической проверки

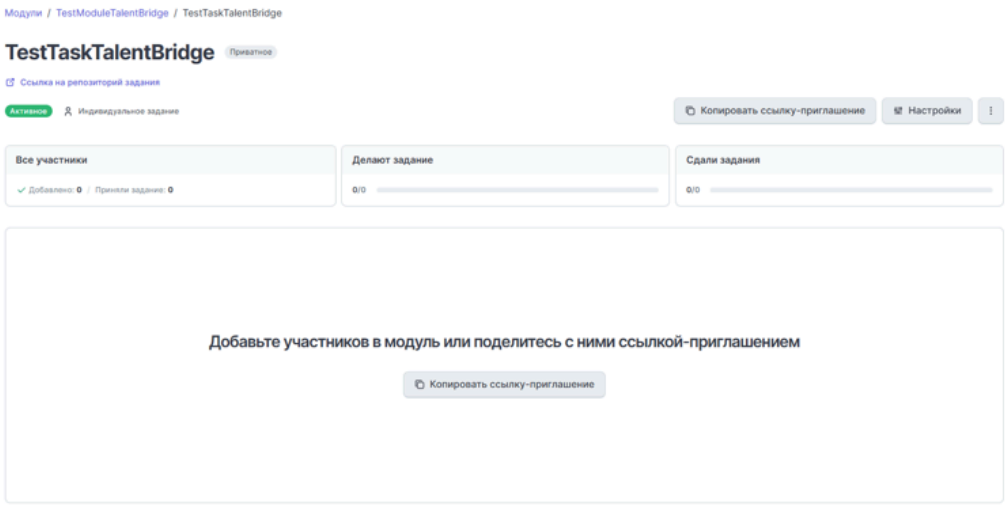


Возможные варианты конструктора автопроверки

9. **Распространение ссылки**

Финальный шаг – нажать **«Копировать ссылку-приглашение»** и отправить ее участникам.

После перехода по ссылке участник принимает задание и получает свою копию репозитория или форк в зависимости от типа задания.
Именно этот шаг завершает организационную часть – задание создано, настроено и готово к выполнению.



Созданное пространство с ссылкой-приглашением на задание

На что необходимо обратить внимание

Репозиторий-шаблон должен быть подготовлен заранее и содержать нужные материалы.

- Тип задания – индивидуальный или групповой – нужно выбирать осознанно, потому что от него зависит модель работы.
- Приватность влияет на видимость и доступ, особенно если задание еще в разработке.
- Дедлайн и его жесткость меняют правила сдачи и редактирования.
- Дополнительные функции – CI/CD, задачи, вики, защита веток – стоит включать только если они реально нужны.

Схема процесса

1. Подготовить структуру модуля.
2. Создать репозиторий-шаблон.
3. Открыть модуль и нажать «Новое задание».
4. Заполнить название, тип, приватность.
5. Установить дедлайн.
6. Выбрать шаблон репозитория.
7. Создать задание.
8. Настроить CI/CD, задачи, вики и защиту веток.
9. Сохранить настройки.

10. Скопировать ссылку-приглашение и разослать участникам.

Процесс настройки CI/CD для заданий в GitVerse Lab

Чтобы настроить **CI/CD для заданий в GitVerse Lab**, нужно связать репозиторий задания с workflow автопроверки и затем включить проверку в самом задании. В GitVerse для этого есть отдельный раздел CI/CD, а в GitVerse Lab прямо поддерживается автопроверка через YAML из шаблона или собственную настройку.

Этап 1. Подготовить репозиторий задания

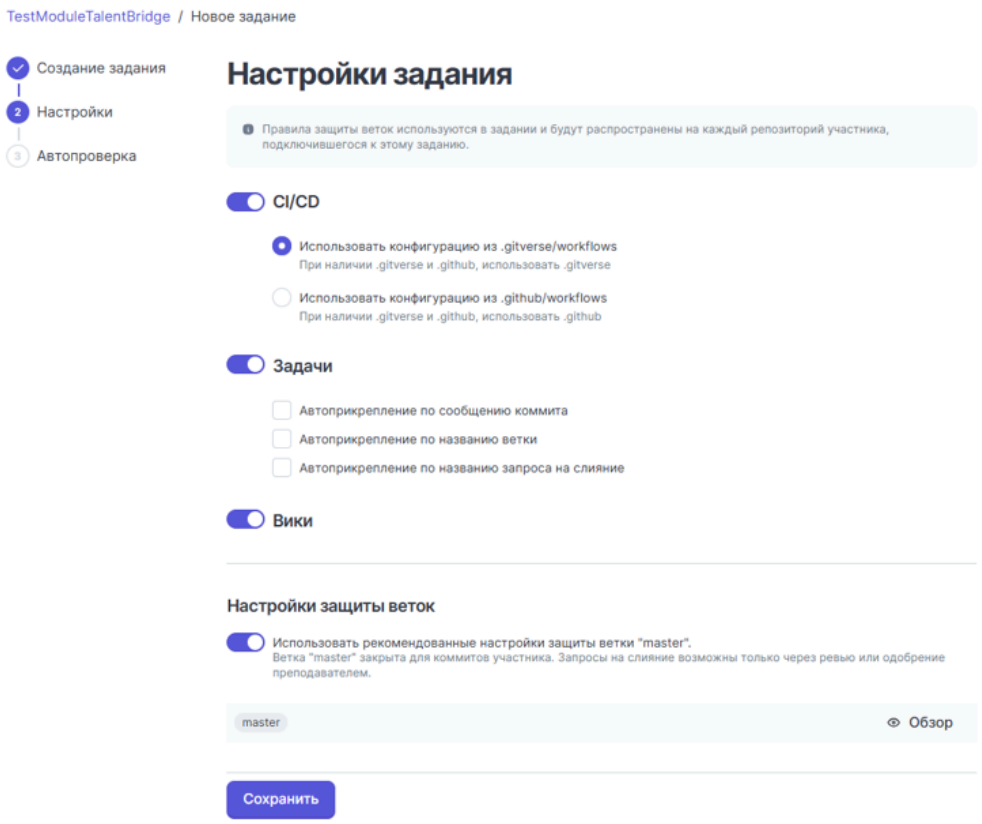
Сначала в репозитории задания должен быть понятный проект с исходным кодом, тестами и конфигурацией для запуска проверок. GitVerse CI/CD работает через workflow, поэтому логика проверки должна быть описана в файлах конфигурации, таких как .gitlab-ci.yml или шаблоне автопроверки.
Если задание создается через GitVerse Lab, репозиторий-шаблон лучше сразу в репозитории подготовить так, чтобы он содержал все, что нужно для запуска тестов и сборки.

Этап 2. Настроить workflow

Далее нужно описать pipeline – что запускать, в каком порядке и на каком раннере. Документация GitVerse CI/CD указывает, что базовый сценарий включает первый workflow со сборкой и тестами, а затем уже можно переходить к более сложным действиям – кешированию, артефактам и уведомлениям.

Этап 3. Включить автопроверку в задании

После подготовки workflow его нужно подключить к самому заданию в GitVerse Lab. В интерфейсе GitVerse можно включить раздел CI/CD для задания и использовать либо собственный YAML автопроверки, либо шаблон из готового проекта.
Основная суть заключается в следующем – кандидат отправляет решение, GitVerse запускает workflow, а результат тестов становится основой для проверки задания.



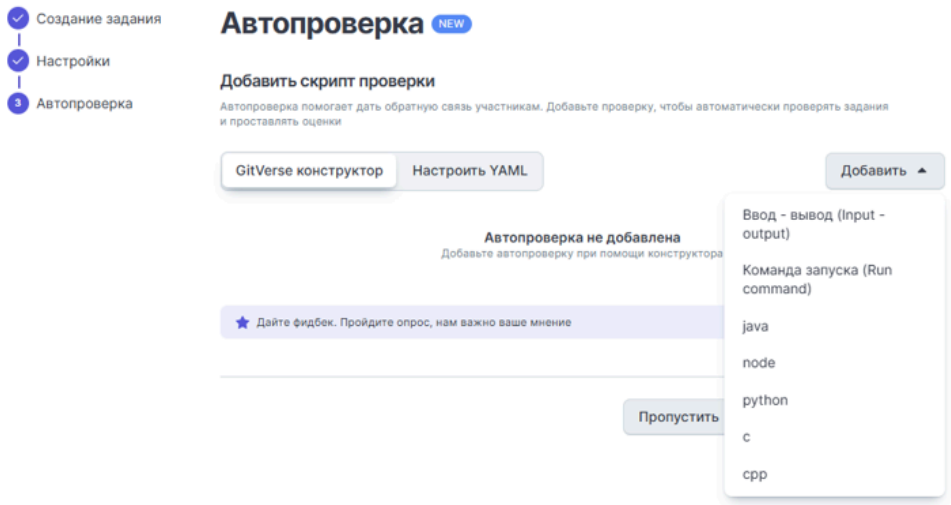
Выбор настроек при создании задания

Этап 4. Проверить триггеры запуска

Важно убедиться, что pipeline стартует в нужный момент – при push, при создании merge request или по другому событию, которое вы используете в учебном процессе. В CI/CD GitVerse workflow запускается на основе триггеров, поэтому неправильная настройка может привести к тому, что автопроверка не сработает.

Этап 5. Настроить проверки

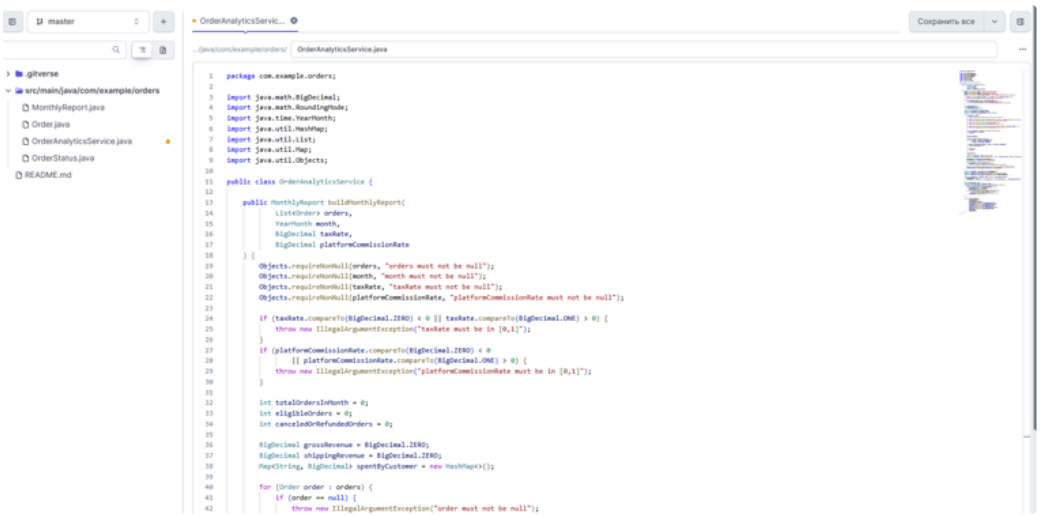
Следующий шаг – сделать сами тесты достаточно точными, чтобы они проверяли задания корректно. В GitVerse CI/CD автопроверка может анализировать код, запускать тесты и собирать результат, а в GitVerse Lab это прямо описано как возможность «настроить автопроверку задания».
Лучше разделять проверки по смыслу – сначала проверка сборки, затем функциональные тесты, затем дополнительные ограничения, если они нужны для задачи.



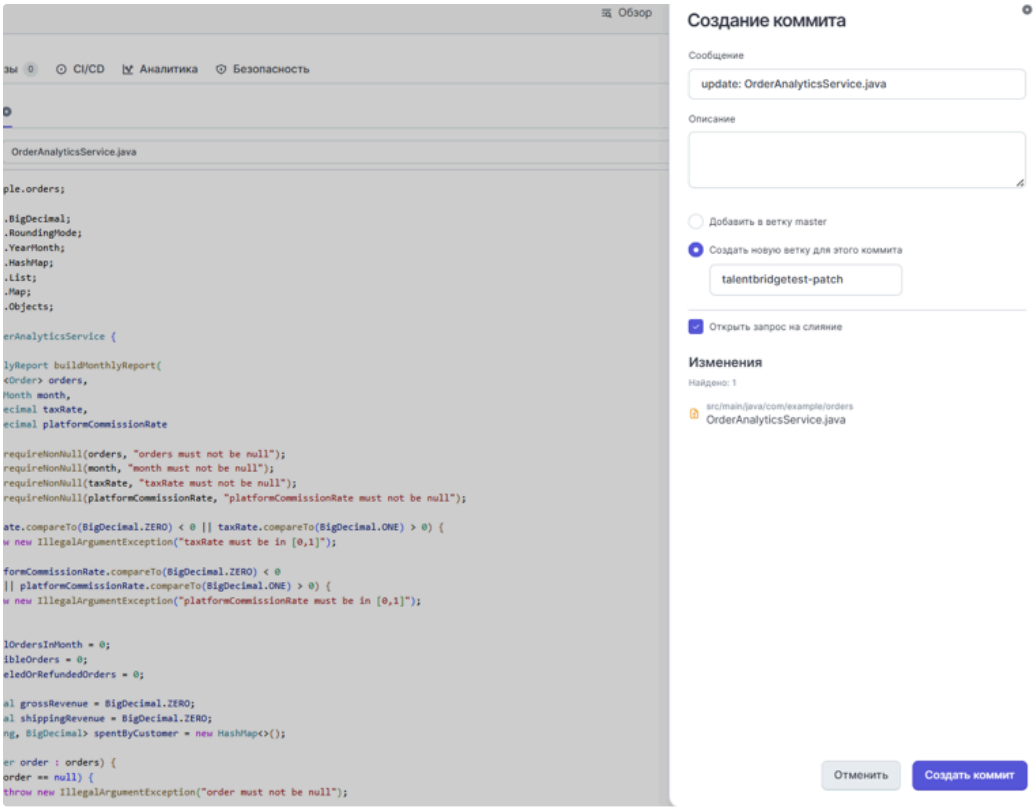
Возможные варианты конструктора автопроверки

Этап 6. Проверить результаты

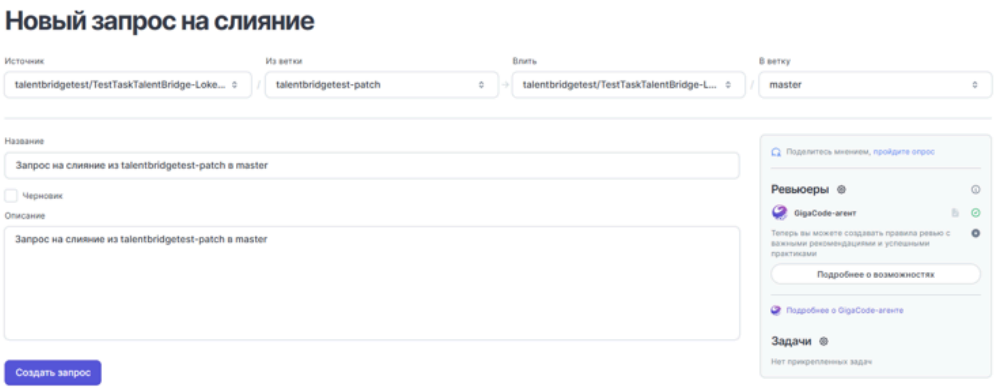
После коммита или отправки решения нужно посмотреть, как отработал pipeline: успешно ли прошли шаги, где возникла ошибка и есть ли артефакты для анализа. GitVerse CI/CD поддерживает запуск на облачных или локальных ресурсах, а также работу с результатами сборки и публикации. При отправке на слияние запускается процесс автопроверок и AI-ревью.



Ввод решения, согласно инструкции, указанной в файле Readme.md



Действия по отправке на проверку задания



Отправка запроса на слияние

На что обратить внимание

- Файл конфигурации CI/CD должен быть в репозитории и соответствовать ожидаемой структуре задания.
- Тесты должны запускаться автоматически и без ручных действий.
- Триггеры запуска нужно согласовать с тем, как студент сдаёт работу.
- Если используете шаблон автопроверки, проверьте, что он подходит именно под ваш тип задания.
- Для учебных задач полезно делать pipeline коротким и предсказуемым, чтобы студент быстро понимал причину ошибки.

Схема процесса

1. Подготовить репозиторий задания с кодом и тестами.
2. Добавить CI/CD-конфигурацию.
3. Определить этапы workflow: build, test, при необходимости lint и report.
4. Подключить автопроверку в GitVerse Lab / SmartClass.
5. Настроить триггеры запуска.
6. Проверить, что результаты отображаются корректно.
7. При необходимости доработать тесты и условия прохождения.

Дополнительные материалы по работе с CI/CD.

- **Синтаксис workflow** – структура workflow-файла;
- **Триггеры запуска workflow** – события инициирующие выполнение workflow;
- **Контекст GitVerse** – контекст GitVerse и работа с ним из workflow;
- **Раннеры** – типы раннеров (облачные/локальные), их регистрация и использование;
- **Секреты и переменные** – использование секретов для хранения конфиденциальных данных (токены, пароли) и переменных;
- **Лимиты** – пользовательские и платформенные ограничения.

Автопроверки и AI-ревью на платформе GitVerse Lab

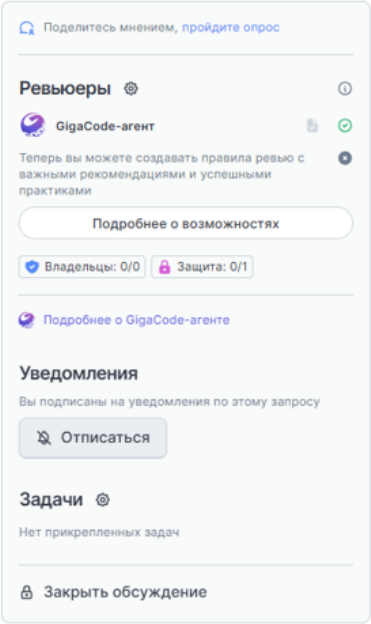
Автопроверка

Сначала в задании настраивают репозиторий-шаблон и добавляют конфигурацию пайплайна, чтобы при отправке решения автоматически запускались сборка и тесты. Дальше GitVerse запускает pipeline по заданному триггеру, а результаты прохождения шагов становятся основой для проверки задания.
Для учебных работ обычно используют короткий и предсказуемый сценарий: сборка, запуск тестов, проверка артефактов и вывод статуса.

AI-ревью

AI-ревью на GitVerse – это отдельная интеллектуальная помощь, которая анализирует код и автоматически помогает находить проблемы в решении.
По сути, AI-агент смотрит на изменения в коде, оценивает их и может подсказать замечания, похожие на ревью от человека.

Может подключаться только владельцем репозитория. Есть возможность настроить автоматический вызов GigaCode-агента, либо с помощью команд.



Колонка GigaCode-агента

Принцип работы

Сначала кандидат отправляет решение в репозиторий задания, затем запускается CI/CD-проверка, после или параллельно с ней может подключаться AI-ревью для анализа качества кода.
Автопроверка отвечает за формальные критерии – сборка, тесты, структура проекта, AI-ревью полезно для более “человеческих” замечаний, таких как читаемость, архитектурные огрехи, потенциальные баги.
Такой подход позволяет не только понять, проходит ли решение тесты, но и увидеть, насколько оно аккуратно и устойчиво написано.

Правила

Вы можете задавать собственные правила для GigaCode-агента, чтобы получать более релевантные и персонализированные предложения по улучшению кода, соответствующие стилю и архитектурным решениям вашего проекта.

Как это работает

GigaCode-агент автоматически учитывает все файлы в формате Markdown (`.md`), расположенные в директории `.gitverse/pr_rules/` **целевой ветки** (в которую выполняется слияние). Эти файлы содержат ваши внутренние стандарты кодирования, архитектурные соглашения и любые другие рекомендации, которые вы хотите применять при анализе запросов на слияние.

Где задаются правила

Чтобы настроить правила:

1. В корне вашей **целевой ветки** создайте директорию `.gitverse/pr_rules/`.
2. Поместите в нее один или несколько файлов с расширением `.md`, например:
   - `.gitverse/pr_rules/coding_standards.md`;
   - `.gitverse/pr_rules/architecture.md`;
   - `.gitverse/pr_rules/security.md`.

Каждый файл должен содержать четкие, понятные инструкции.

Рекомендации по написанию правил

Чтобы правила приносили максимальную пользу и корректно интерпретировались GigaCode-агентом, следуйте этим рекомендациям.

1. Формулируйте цель и задачу

Каждое правило должно решать **конкретную проблему** или отвечать на **четкий вопрос**:

- ❌ «пишите хороший код» – слишком абстрактно;
- ✅ «все HTTP-запросы к внешним API должны логироваться с указанием URL и статуса ответа» – конкретно и проверяемо.

Описывайте проблему кратко, но достаточно подробно, чтобы любой разработчик понял, **почему** это правило важно.

2. Указывайте технический контекст

Для повышения точности укажите:

- **язык программирования**, фреймворк или технологию (например, Python 3.10+, React 18, Django 4.2);
- **версии инструментов**, если они критичны (например, «Требуется Node.js ≥ 18.0»);
- **ограничения среды**: ОС, runtime, зависимости, CI/CD-окружение и т.д.

Пример:

> *Для всех микросервисов на Python (версия ≥ 3.9) в Docker-образах на базе Alpine Linux:*
>
> - *используйте* `asyncio` *вместо многопоточности;*
> - *не устанавливайте пакеты через* `pip` *глобально – только в виртуальное окружение.*

3. Приводите примеры «до / после»

Лучше всего правила работают в **паттерн-ориентированном формате**:

Проблема → **Решение** → **Пример кода**.

На что обращать внимание

- Конфигурация CI/CD должна быть в шаблоне задания и запускаться без ручных действий.
- Автопроверка должна быть быстрой, иначе кандидату сложно итеративно исправлять ошибки.
- AI-ревью лучше использовать как дополнительный слой, а не как единственный критерий оценки.
- Для практических заданий полезно разделять техническую проверку и смысловую оценку кода.

Пример:

Паттерн: защита от SQL-инъекций в Django

Проблема: использование сырых SQL-запросов без параметризации может привести к уязвимостям.

Решение: всегда используйте ORM или параметризованные запросы через `cursor.execute()`.

Пример до:

```
query = f"SELECT * FROM users WHERE email = '{email}'"cursor.execute(que
```

Пример после:

```
cursor.execute("SELECT * FROM users WHERE email = %s", [email])
```

4. Описывайте ожидаемое поведение

Четко формулируйте, **что должно быть**, а не только **чего нельзя делать**:

- ❌ «не используйте `var` в JavaScript»;
- ✅ «все переменные в проекте должны объявляться с `const` или `let` в соответствии с ES2022».

5. Указывайте предпочтительные инструменты и методы

Перечислите:

- разрешенные/запрещенные библиотеки;
- предпочтительные паттерны проектирования;
- требования к асинхронности, обработке ошибок, типизации и т.д.

Пример:

> - *используйте `Zod` для валидации входных данных в API;*
> - *избегайте глобального состояния в React-компонентах;*
> - *все асинхронные операции должны быть обернуты в `try/catch` с логированием ошибок.*

6. Добавляйте тестовые сценарии

Если правило связано с поведением, которое проявляется только в определенных условиях, опишите **тестовые кейсы**:

> ***Сценарий**: При загрузке файла размером >100 МБ через форму `/upload` сервер не должен падать. **Ожидаемое поведение**: Запрос отклоняется с кодом `413 Payload Too Large`.*

Следуя этим рекомендациям, вы поможете GigaCode-агенту давать более точные, полезные и релевантные предложения.

Ограничения

1. Если в открытый запрос на слияние добавились коммиты для обновления комментариев от GigaCode-агента, отправьте нужную команду повторно.
2. В закрытых запросах (закрытые без слияния и успешно выполненные) комментарии от GigaCode-агента не обрабатываются.

3. Если GigaCode-агент был включен в репозитории после создания запроса на слияние, то комментарии от GigaCode-агента не будут созданы автоматически.

Информация по правилам и ограничениям взята с https://gitverse.ru/docs/ai/gigacode-agent/#%D0%BF%D1%80%D0%B0%D0%B2%D0%B8%D0%BB%D0%B0

Процесс автопроверки и AI-ревью на платформе GitVerse Lab

1. **Получить задание**

Кандидат сначала принимает задание по приглашению и получает доступ к своему варианту репозитория.
После этого он видит материалы задания, кодовый шаблон и условия проверки.
На этом этапе важно прочитать требования, чтобы понимать, что именно будет проверяться автоматически и что может попасть в AI-ревью.

2. **Внести решение**

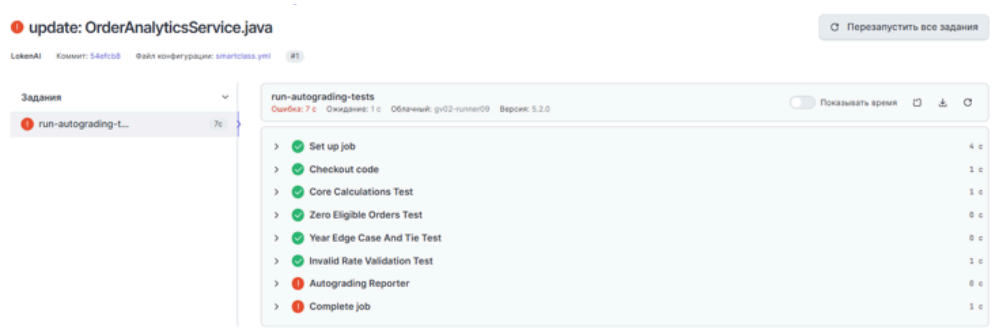
Кандидат вносит изменения в код, добавляет или исправляет функции, тесты и конфигурацию, если это требуется условиями задания.
Обычно лучше делать это небольшими шагами, чтобы было проще понять, на каком коммите возникла ошибка.
Если в задании есть автотесты, именно их прохождение станет основой для успешной сдачи.

3. **Отправить изменения**

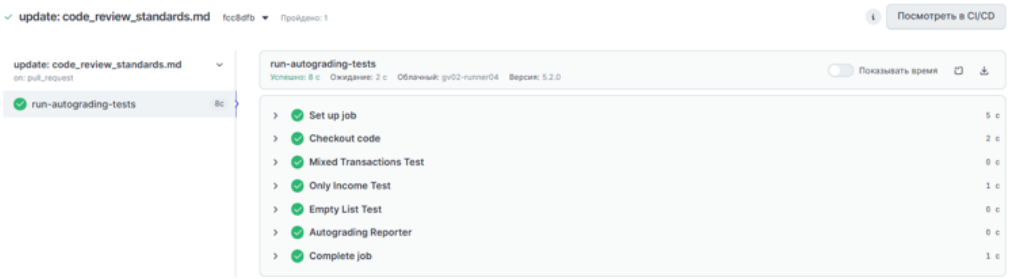
После локальной проверки код отправляется в репозиторий задания.
Именно этот момент обычно запускает проверочный pipeline, если он настроен в задании.
Для кандидата это значит, что дальше оценка идет уже автоматически, без ручного запуска со стороны преподавателя.

4. **Запуск автопроверки**

GitVerse запускает **CI/CD-пайплайн**, где выполняются сборка, тестирование и другие заданные этапы проверки.
В документации GitVerse прямо указано, что для задания можно использовать YAML автопроверки из шаблона или настроить проверку самостоятельно.
Если сборка падает или тесты не проходят, задание не считается успешно выполненным.



Пример автопроверки задания, закончившейся ошибкой



Пример автопроверки задания, закончившейся успешно

5. **Анализ результата**

После завершения pipeline кандидат смотрит статус проверки и, если есть ошибка, исправляет код.
Этот цикл обычно повторяется несколько раз: отправка кода, запуск проверки, просмотр результата, исправление.
Для кандидата ключевая цель на этом шаге – добиться не только успешной сборки, но и полного прохождения всех тестов.

6. **Получить AI-ревью**

Параллельно с формальной автопроверкой может работать **AI-обзор**: он анализирует прикрепленные к задаче изменения и делает комплексный разбор кода.
AI-обзор выполняется **в фоне**, поэтому страницу задачи можно закрыть, а результат потом посмотреть отдельно.
Это полезно, потому что AI может указать на архитектурные проблемы, неочевидные ошибки и слабые места в читаемости кода.

gigacode-agent Агент комментарий минуту назад

Ревью

Несколько ключевых моментов, которые помогут в процессе проверки:

Уровень сложности ревью: 4

Нет релевантных тестов

Проблем с безопасностью не выявлено

Рекомендовано к рассмотрению

▸ **Возможная проблема**
Метод не проверяет, что rate не является NaN или бесконечным значением, что может привести к некорректным вычислениям.

▸ **Возможная проблема**
При расчёте averageCheck не учитывается возможность деления на ноль, хотя перед этим есть проверка eligibleOrders > 0, однако логика дублируется и может быть упрощена.

▸ **Code Quality**
В цикле определения основного клиента при равенстве сумм сравнивается идентификатор клиента как строка, что может привести к неочевидному поведению; возможно, требуется уточнение бизнес-логики.

Пример AI-ревью

Улучшение ✨

Изучите дополнительные предложения по коду:

Категория	Предложение	Импакт

Критический дефект	▼ Корректная сортировка при равенстве
	При обновлении `topCustomerId` при равных суммах нужно корректно обрабатывать случай, когда текущий `topCustomerId` — "N/A", чтобы не пропустить первого клиента.
	src/main/java/com/example/orders/OrderAnalyticsService.java [107-111]
	`} else if (currentAmount.compareTo(topCustomerAmount) == 0`
	~~`&& !"N/A".equals(topCustomerId)`~~
	`&& entry.getKey().compareTo(topCustomerId) < 0) {`
	`topCustomerId = entry.getKey();`
	▼ Важность предложенного улучшения[1-10]: 8
	—
	Причина: Условие !"N/A".equals(topCustomerId) мешает обновлению topCustomerId при первом клиенте с ненулевой суммой, если суммы равны. Удаление этого условия исправляет логическую ошибку, позволяя корректно выбирать клиента с лексикографически меньшим ID. Это важная ошибка в логике сортировки.
		Средний

	▸ Избежание деления на ноль	Средний

Пример возможных улучшений первая часть

gigacode-agent Агент комментарий 2 минуты назад

Улучшение ✨

Изучите дополнительные предложения по коду:

Категория	Предложение	Импакт
Критический дефект	▶ Корректная сортировка при равенстве	Средний
	▼ Избежание деления на ноль	Средний

При делении для вычисления среднего чека необходимо проверить, что делитель больше нуля, чтобы избежать деления на ноль, даже если логика предполагает иное.

src/main/java/com/example/orders/OrderAnalyticsService.java [95-98]

```
-BigDecimal averageCheck = BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP);
-if (eligibleOrders > 0) {
-    averageCheck = grossRevenue.divide(BigDecimal.valueOf(eligibleOrders), 2, RoundingMode.H
-}
+BigDecimal averageCheck = eligibleOrders == 0 ? BigDecimal.ZERO : grossRevenue.divide(BigDec
+averageCheck = averageCheck.setScale(2, RoundingMode.HALF_UP);
```

▼ Важность предложенного улучшения[1-10]: 7

—

Причина: Существующий код уже проверяет eligibleOrders > 0, поэтому деление на ноль невозможно. Однако предложение предлагает более компактную форму, что улучшает читаемость без изменения логики. Это умеренное улучшение.

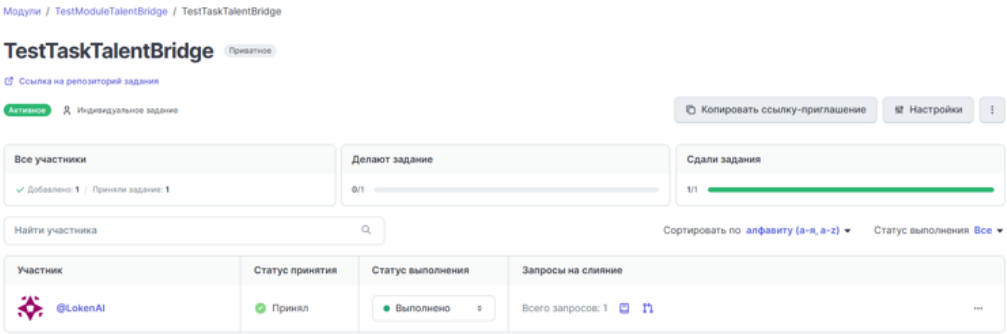
Пример возможных улучшений вторая часть

7. **Использовать замечания AI**

Кандидат читает замечания AI-ревью и решает, какие из них требуют исправления.
Важно воспринимать AI-обзор как дополнительную обратную связь, а не как единственный критерий качества решения.
Если AI указывает на повторяющийся код, плохую структуру или риск багов, лучше внести улучшения до финальной сдачи.

8. **Завершить сдачу**

Когда автопроверка проходит успешно и код соответствует замечаниям, кандидат завершает выполнение задания.
В идеале итоговый код должен быть не только технически корректным, но и достаточно чистым для положительного AI-анализа.
Такой подход показывает, что решение прошло и формальную, и смысловую проверку.



Итоговый результат

На что смотреть особенно внимательно

- Проверяй, что код действительно запускается в той же среде, что и автопроверка.
- Следи за тестами: именно они чаще всего являются решающим фактором.
- Не игнорируй AI-комментарии, даже если формально все тесты зеленые.
- Если задача использует шаблон автопроверки, проверь, что структура проекта совпадает с ожиданиями шаблона.

Краткая схема

1. Принять задание.
2. Изучить требования.
3. Написать и отправить код.
4. Дождаться запуска CI/CD.
5. Исправить ошибки по результатам автопроверки.
6. Посмотреть AI-ревью.
7. Внести улучшения.
8. Повторить цикл до успешной сдачи.

Как назначить задание участникам модуля Gitverse Lab

1. **Откройте нужный модуль**

Войдите в GitVerse и перейдите в тот **модуль**, где будет размещено задание.
Убедитесь, что вы находитесь именно внутри нужного курса или лабораторного модуля, а не в обычном репозитории.
Это важно, потому что задание привязывается к модулю и потом отображается участникам именно в его рамках.

2. **Создайте новое задание**

Нажмите кнопку **«Новое задание»** внутри модуля.
После этого откроется форма создания задания, где нужно указать основные параметры. На этом этапе вы фактически создаете объект задания, к которому позже будут подключаться участники.

3. **Заполните основные параметры**

Введите **название задания** и выберите уровень приватности – публичное или приватное. Публичное задание видно всем участникам модуля, а приватное – только владельцам организации и участнику, который примет задание.
Здесь же выберите **тип задания**: индивидуальное или групповое.
Для индивидуального задания каждый участник работает отдельно, а для группового можно задать команды и ограничения по их размеру.

4. **Привяжите репозиторий-шаблон**

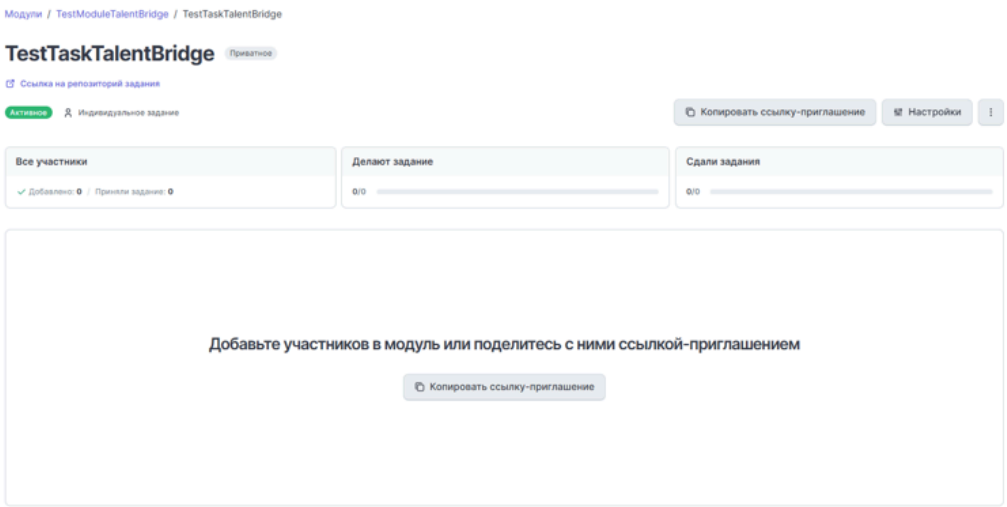
Для каждого задания рекомендуется подготовить отдельный **репозиторий-шаблон** с исходными материалами.
Именно из него GitVerse создаст копию или форк для участника после принятия задания. Если шаблон еще не готов, его лучше создать заранее и наполнить кодом, README и другими нужными файлами.

5. **Сохраните задание**

После заполнения формы нажмите **«Сохранить»**.
В некоторых сценариях можно сразу настроить проверку, а можно временно пропустить этот шаг и вернуться к нему позже.
Это удобно, если вы хотите сначала опубликовать задание, а затем донастроить автопроверку или CI/CD.

6. **Копируйте ссылку-приглашение**

После сохранения нажмите **«Копировать ссылку-приглашение»**.
Эта ссылка нужна, чтобы участники могли принять задание и присоединиться к нему. Ссылку можно отправить через LMS, чат группы, электронную почту или любую другую удобную для вас площадку.



Пространство с ссылкой-приглашением на задание

7. **Разошлите задание участникам**

Отправьте ссылку всем нужным участникам модуля.
После перехода по ней участник принимает задание и получает доступ к своей версии репозитория.
Если задание индивидуальное, каждому создается отдельная персональная копия; если групповое – доступ получают участники команды.

8. **Проверьте статус принятия**

После рассылки отправитель может отслеживать, кто уже принял задание, а кто еще нет. Это помогает контролировать прогресс группы и при необходимости напоминать участникам о необходимости присоединиться.
Для публичных заданий также удобно видеть общий список и статус выполнения по участникам модуля.

На что обратить внимание

- Выбирайте **индивидуальный или групповой** формат заранее, потому что тип задания потом лучше не менять.
- Если задание должно быть доступно всем участникам модуля, используйте **публичный** режим.
- Если задание находится в разработке или должно быть доступно ограниченно, лучше использовать **приватный** режим.
- Для задания с автопроверкой подготовьте репозиторий так, чтобы он сразу запускался в CI/CD.

Короткая схема

1. Открыть модуль GitVerse Lab.
2. Нажать **«Новое задание»**.
3. Указать название, приватность и тип.
4. Подготовить и привязать репозиторий-шаблон.
5. Нажать **«Сохранить»**.
6. Скопировать ссылку-приглашение.
7. Отправить ее участникам модуля.
8. Отслеживать, кто принял задание.

Алгоритм создания репозитория с скрытыми тестами и как его настроить

1. **Создайте два репозитория**

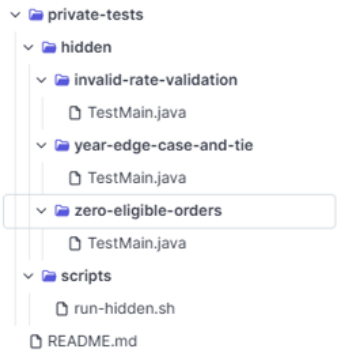
Сначала сделайте основной репозиторий задания, например ExampleTask, и отдельный приватный репозиторий TestsForExample.
В ExampleTask будут лежать исходники задания и открытые тесты, а в TestsForExample – только скрытые сценарии и скрипт запуска.

2. **Настройка приватности**

Репозиторий TestsForExample должен быть **private**.
Иначе участник сможет открыть его содержимое напрямую и скрытые тесты перестанут быть скрытыми.

3. **Создание файловой структуры в репозитории в TestsForExample**

Примерная структура показана ниже:



Пример структуры в репозитории TestsForExample

hidden/ содержит сами проверки, а scripts/run-hidden.sh отвечает за компиляцию и запуск.

4. **Заполнение скрытых тестов**

В каждом TestMain.java следует держать один сценарий, к примеру:

- zero-eligible-orders – проверка пустой eligible-выборки;
- year-edge-case-and-tie – проверка фильтра по году и tie-break;
- invalid-rate-validation – проверка исключения на невалидную ставку.

Это удобнее, чем хранить все проверки в одном файле, потому что каждый случай можно независимо поддерживать и менять.

5. **Добавление run-hidden.sh**

Скрипт должен:

- найти папку с исходниками student repo;
- скомпилировать код студента вместе с нужным TestMain.java;
- запустить тест;
- выйти с ошибкой, если тест не прошел.

Именно этот скрипт будет вызываться из workflow в ExampleTask.

```bash
#!/usr/bin/env bash
set -euo pipefail

if [ -d "Task/src/main/java" ]; then
  SRC="Task/src/main/java"
elif [ -d "source_files/Task/src/main/java" ]; then
  SRC="source_files/Task/src/main/java"
elif [ -d "src/main/java" ]; then
  SRC="src/main/java"
else
  echo "Source folder not found"
  exit 1
fi

run_test() {
  local test_dir="$1"
  local out_dir="$2"

  rm -rf "$out_dir"
  mkdir -p "$out_dir"

  javac -d "$out_dir" $(find "$SRC" -name "*.java") "$test_dir/TestMain.java"
  java -cp "$out_dir" TestMain
}

run_test "private-tests/hidden/zero-eligible-orders" "/tmp/ag-hidden1/out"
run_test "private-tests/hidden/year-edge-case-and-tie" "/tmp/ag-hidden2/out"
run_test "private-tests/hidden/invalid-rate-validation" "/tmp/ag-hidden3/out"
```

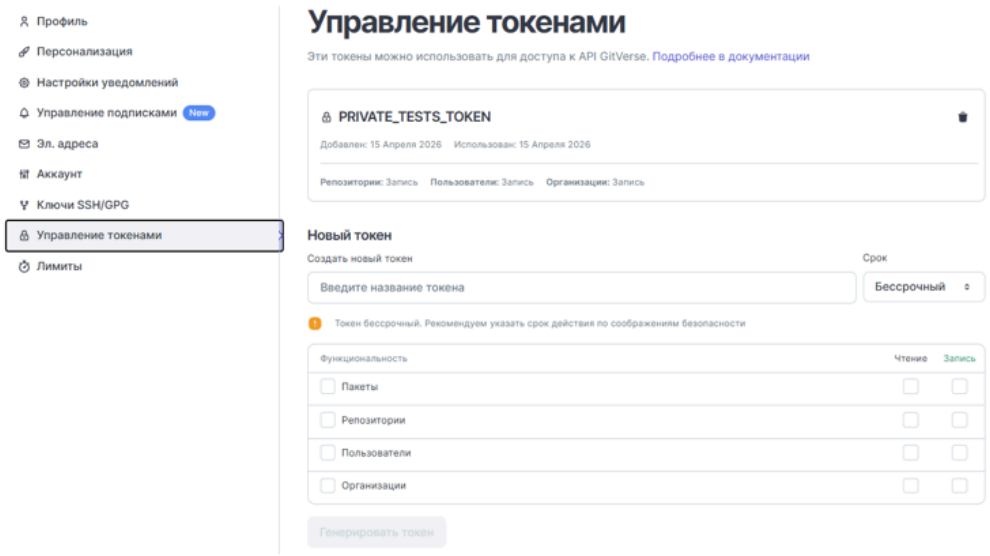
Пример скрипта

6. **Создание PAT**

Чтобы ExampleTask мог прочитать TestsForExample, нужен **Personal Access Token** с доступом к приватному репозиторию.

Для генерации токена, нужно перейти в настройках профиля в раздел “Управление токенами”.

Секрет необходимо хранить в ExampleTask как PRIVATE_TESTS_TOKEN, а сам токен не публикуется в коде.



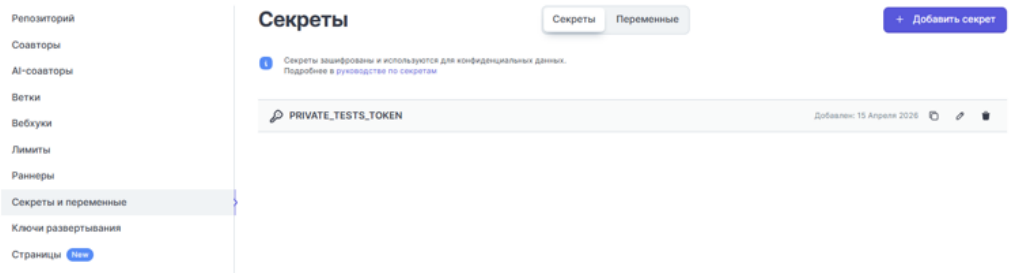
Генерация токена

7. **Добавление secret в ExampleTask**

Для создания секретов нужно перейти в настройки репозитория в раздел “Секреты и переменные”:

- имя: PRIVATE_TESTS_TOKEN
- значение: PAT, который имеет доступ к TestsForExample

Это нужно, чтобы workflow мог аутентифицироваться при скачивании приватного репозитория.



Создание секрета

8. **Настройка smartclass.yml**

В ExampleTask workflow должно быть:

1. checkout student code;
2. скачать TestsForExample через actions/checkout;
3. запустить run-hidden.sh из приватного репозитория.

Пример шага:

```
name: Checkout private tests
  uses: actions/checkout@v4
  with:
    repository: talentbridgetest/TestsForExample
    ref: main
    token: ${{ secrets.PRIVATE_TESTS_TOKEN }}
    path: private-tests
```

9. **Укажите правильный репозиторий**

Значение repository должно быть в формате {owner}/{repo} и без домена.
Если GitVerse показывает другой namespace в URL, используй именно его, но только в двухчастном формате owner/repo.

10. Подключение запуска скрытых тестов

После checkout приватного репозитория запустите:

```
name: Run hidden tests
  run: bash private-tests/scripts/run-hidden.sh
```

Так workflow выполнит скрытые проверки, но не покажет их содержимое в репозитории, который виден участникам.

11. Проверка права доступа

Если checkout не работает, проверьте:

- PAT действительно имеет доступ к TestsForExample;
- secret создан именно в ExampleTask;
- репозиторий TestsForExample приватный и не пустой;
- имя ветки в приватном репо совпадает с ref: main или другим заданным значением.

12. Проверка итогового сценария

После настройки в ExampleTask участник будет видеть только:

- открытые тесты;
- факт запуска CI.

Сами скрытые тесты останутся в TestsForExample и будут запускаться только workflow'ом с токеном.